

Weekly Project Road Map

Week 2 — Menu System Design & Customization Architecture

Objective

The goal of this week is to design and implement the **core data model and logic for menu management**, including dynamic customization options defined by caretakers.

Required Work

Students must design and implement the following components:

1. Menu Item Management (Caretaker Side)

- Caretakers must be able to:
 - create new menu items,
 - update existing menu items,
 - delete menu items if needed.
- Each menu item must include:
 - name/title,
 - price,
 - description,
 - image upload (mandatory).

2. Image Handling

- Menu item images must be:
 - uploaded via the application,
 - stored properly (file system or database),
 - retrievable and displayed in the UI.

3. Customization System (Core Requirement)

Students must design a **dynamic customization system**, not a hardcoded one.

Each menu item must support:

- removable ingredients (boolean-based options),
- optional additions (extra ingredients),
- grouped options (e.g., sauces, side selections),
- optional price modifications per customization.

Customization must:

- be defined by the caretaker,
- be stored in the database,
- support multiple option groups,
- be reusable and extendable.

4. User Menu View

Users must be able to:

- view menu items,
- see menu details clearly,
- see associated caretaker information.

At this stage, rating values may be placeholder or optional (full implementation in Week 4).

Expected Outcome

By the end of Week 2:

- Menu system must be fully functional,
- Caretakers must be able to create menu items with images,
- Customization structure must be implemented and stored dynamically,
- Users must be able to view menus and their structure.

Week 3 — Cart System, Order Processing & Payment Simulation

Objective

The goal of this week is to implement the **complete order flow**, from selecting menu items to completing a simulated payment.

Required Work

1. Cart System Implementation

Students must implement a fully functional cart system where users can:

- add menu items to the cart,
- select customization options during item selection,
- update item quantities,
- remove items from the cart.

The cart must:

- dynamically calculate total price,
- include customization-based price changes,
- persist data during the session (or database).

2. Order Data Model

Students must design the order-related schema, including:

- orders,
- order items,
- selected customization options per item,
- total price,
- timestamps.

3. Payment Simulation

Students must implement a payment page that:

- collects card information,
- displays full order summary,
- validates input (basic validation is sufficient),
- simulates a successful transaction.

Real payment systems are not required and must not be used.

4. Order Completion Flow

After payment:

- an order must be created in the database,
- all selected items and customizations must be stored,
- order status must be initialized (e.g., pending/completed).

Expected Outcome

By the end of Week 3:

- Users must be able to complete an order from start to finish,
- Cart must function correctly with customization logic,
- Orders must be correctly stored and retrievable.

Week 4 — Integration Features & System Intelligence

Objective

This week focuses on integrating external services and implementing system-level intelligence such as filtering, rating, communication, and logging.

Required Work

1. Google Maps API Integration

Students must:

- obtain user location,
- calculate distances between users and caretakers,
- filter restaurants based on proximity.

All location-based decisions must rely on **Google Maps API**.

2. Rating System (Dual Structure)

Students must implement:

- menu item ratings,
- caretaker ratings.

Requirements:

- ratings must be linked to completed orders,
- users must be able to leave comments,
- average ratings must be calculated dynamically.

3. Email System

Students must implement a working email system that:

- sends order details to users,
- sends order details to caretakers,
- includes meaningful order information.

4. Logging System

Students must design a logging mechanism that records:

- authentication events,
- order creation,
- payment actions,
- rating submissions,

- errors and system events.

5. Admin Logging Interface

Admins must:

- access logs via a UI page,
- review system activities.

Expected Outcome

By the end of Week 4:

- Location-based filtering must work correctly,
- Rating system must be functional,
- Emails must be sent automatically,
- Logs must be stored and viewable.

Week 5 — Finalization, Document Generation & System Refinement

Objective

The goal of this week is to complete all remaining features, improve quality, and prepare the system for evaluation.

Required Work

1. Dynamic Document Generation

Students must implement:

- **receipt generation** for each order,
- **agreement PDF generation.**

Requirements:

- documents must be generated dynamically,
- must include order-specific data,
- must not be static templates.

2. Table Enhancements

All tables in the system must:

- support filtering,
- support pagination.

3. UI/UX Improvements

Students must:

- improve visual quality,
- ensure consistency,
- apply unique branding,
- refine layout and usability.

4. System Testing & Bug Fixing

- identify and fix bugs,
- test all major flows:
 - login,
 - menu,
 - cart,
 - payment,
 - rating,
 - logging.

5. Final Preparation

Students must prepare:

- project report,
- database scripts,
- demo scenario.

Expected Outcome

By the end of Week 5:

- The system must be fully functional,
- All required features must be complete,
- The project must be ready for live demonstration.